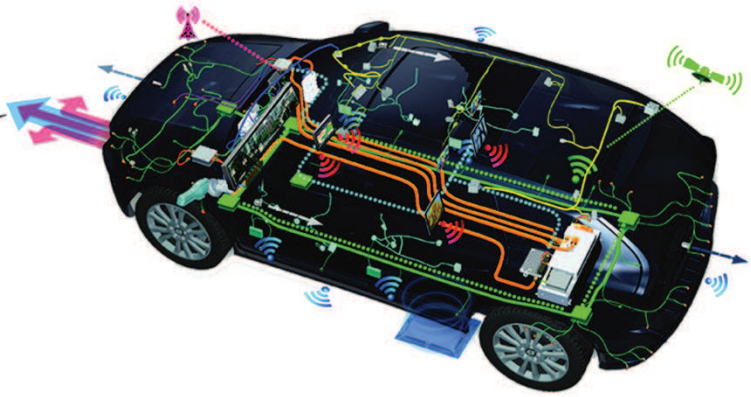
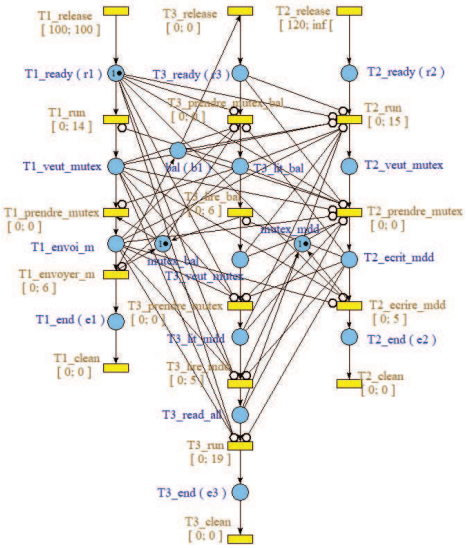
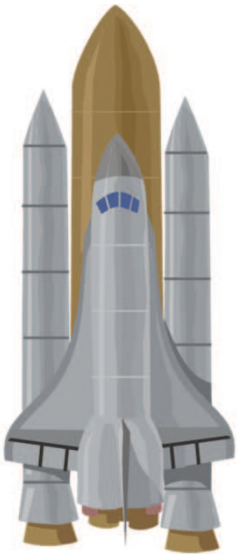
Ecole Nationale Supérieure
de Mécanique et d'Aérotechnique
Téléport 2, 1 avenue Clément Ader - B.P. 40109 - 86961 FUTUROSCOPE Cedex
(FRANCE)



Validation Temporelle
Response-Time Analysis, Model Checking





Emmanuel Grolleau
grolleau@ensma.fr
Bureau B414

Janvier 2025

TD VAT flottant : Complexité algorithmique

La complexité représente la substantifique moëlle d'un problème. Chaque problème possède sa propre complexité intrinsèque.

1 Complexité d'un algorithme vs. d'un problème

Soit T un tableau d'entiers de taille n, **non trié**. Voici un algorithme testant si le tableau possède une valeur E donnée.

```
Inclue (T : tableau de n entiers, E : entier)
-- Renvoie vrai ssi E<max(T)
Pour i de 1 à n
      Si T(i)=E retourner vrai
Retourner faux
```

Ici, ce qui peut varier est n, la taille du tableau : 10 ou 1 milliard ? Les valeurs numériques sont assumées codées sur des entiers logeant dans des registres, 32 ou 64 bits par exemple. Par conséquent, nous supposons que les opérations de comparaison (instruction dans la boucle pour, ou même traitement de l'indice de boucle) ont des durées constantes. La taille de l'entrée est (2+n)*taille entier, taille du tableau, entier E, et tableau lui-même. Tout étant constant à part n, nous disons donc que l'entrée a une taille n donnée par la taille du tableau.

a. Quelle est la complexité de cet algorithme en fonction de n ?

Maintenant observons le problème, peut-on faire mieux ? Par exemple, la mémoire cache effectue cette recherche de clé à chaque requête à la RAM. Pensez-vous qu'elle parcoure les valeurs de clés stockées en cache une par une ? Non, une mémoire associative permet, da façon matérielle, une recherche en parallèle sur toutes les clés. Une mémoire associative n'est pas une architecture Von Neumann. Il est complexe de modéliser un ordinateur complet, Alan Turing a proposé une abstraction très simple, que l'on nomme **Machine de Turing** :

Une machine de Turing est définie par une tête de lecture/écriture sur un ruban infini à gauche et à droite de la tête. Le ruban est constitué de cases pouvant contenir des symboles de l'alphabet ou le symbole blanc (noté b ou ε), qui remplit toutes les cases non initialisées du ruban. Ce ruban correspond à une mémoire séquentielle. La tête suit un algorithme qui est en fait une machine à états, qui dépend du symbole lu, et de l'état courant. En fonction de cela, la transition donne le symbole à écrire, et l'action à effectuer : un pas à gauche L, ou un pas à droite R. Un état spécial, Halt, est atteint lorsque c'est un état final de la machine à états, ou bien si aucune transition de l'état courant avec le symbole lu n'existe (cela correspondrait à un plantage). On peut par exemple imaginer un comptage en base 1, comme sur cet exemple, tiré d'un TD du Dr. Steven J. Zeil.

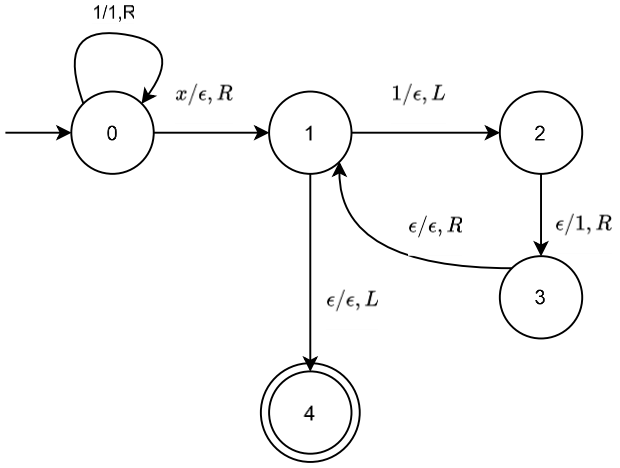


Figure 1 : Système d'instructions d'une machine de Turing additionnant deux nombres en base 1. L'entrée est donnée sur le ruban à droite de la tête, sous forme 111x11 par exemple pour 3+2, ici nous devons obtenir 11111.

b. Vérifier que pour l'entrée 111x11, avec la tête sur le premier 1, on obtient bien 11111 avec la tête sur le dernier 1.
c. Qu'ajouter pour que la tête soit positionnée sur le premier 1 ?
d. Que se passe-t-il si on a un symbole non reconnu en entrée ?
e. Supposons que nous voulions faire un programme Incrémenter, mais d'un entier donné en décimale. La tête est initialement placée sur le chiffre de poids faible, à droite du ruban. Le programme doit incrémenter l'entier et placer à la fin la tête sur le chiffre de poids faible. Donner le programme sous forme d'un automate comme sur la Figure 1.

La Machine de Turing ne va pas nous servir à calculer une complexité, car l'absence d'un accès direct à la mémoire (une bande est une mémoire séquentielle) va augmenter le nombre d'opérations nécessaires. Cependant, elle est équivalente en puissance d'expression à une Random Access Machine qui représente registres, RAM et opérations de base d'un microprocesseur. C'est sur ce type de modèle que la complexité d'un programme est assumée.

La Machine de Turing a notamment été utilisée pour étudier la décidabilité. Un problème est décidable si une réponse au problème peut être donnée en temps fini. Elle sert à démontrer par l'absurde que le problème de l'arrêt d'un programme est indécidable.

f. Supposons maintenant que le tableau d'entrée est trié par ordre croissant. Quelle est la complexité de recherche d'un élément ?
g. Notez que la fonction Inclue fonctionne sur un tableau trié, mais que dire de la complexité de cet algorithme par rapport à la complexité du problème ?

Visiter le site https://www.bigocheatsheet.com/ qui met en évidence des problèmes et complexités d'opérations classiques.

2 Classes de complexité des problèmes

Comment savoir si un problème est intrinsèquement complexe, par exemple s'il ne permet pas de solution polynomiale sur une architecture Von Neumann ?

Classification par classes de complexité P (polynomial), NP (non-déterministe polynomial), co-NP (dont le complémentaire est NP) ou NP^{coNP}, $CoNP^{NP}$, $NP^{CoNP\wedge NP}$ etc.

Considérons les problèmes de décision, c'est-à-dire les algorithmes qui répondent OUI/NON, comme c'était le cas pour l'algorithme Inclue. Notons qu'un problème de recherche se transforme en problème de décision, ainsi trouver le plus court chemin dans un graphe, se transforme en existe-t-il un chemin de longueur <= k ? Nous voulons classifier ces problèmes suivant leur complexité.

Problème de référence : SATISFIABILITY (SAT), et même 3SAT sa version limitée à la satisfiabilité de formule booléenne constituées de conjonctions de disjonctions de 3 littéraux.

Problème **3SAT** : soit une formule booléenne constituée d'une conjonction de disjonctions de 3 littéraux (exemple : (a ou non b ou c) et (non a ou non b ou c)). Existe-t-il une valuation des variables booléennes telle que la formule est vraie ?

Réponse : il nous faut pour le problème général tester toutes les valuations possibles.

a. Utiliser un arbre représentant toutes les valuations possibles pour (a ou non b ou c) et (non a ou non b ou c). Au premier niveau on représente a, qui vaut vrai ou faux, b au second, et c au troisième niveau, toutes les valeurs possibles étant représentées au niveau de chaque feuille.
b. Quelle est la taille d'un arbre d'une formule constituée de n variables ?
c. Si la réponse est OUI, quelle est la complexité de la vérification de cette réponse ?
d. Si la réponse est NON, quelle est la complexité de la vérification de cette réponse ?

3SAT est l'un des problèmes **NP-complets**. Il en existe des centaines, recensés notamment dans [1]. Un problème est NP si exhiber un exemple permet de démontrer que la réponse est OUI, alors que démontrer que la réponse est NON requière l'arbre entier.

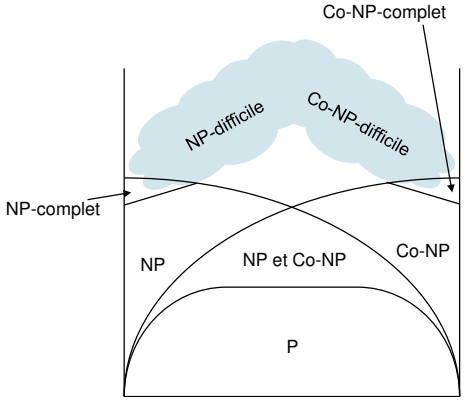


Figure 2 : une représentation possible des classes de complexité P, NP et Co-NP

Les problèmes **Co-NP** sont complémentaires dans le sens où dire NON requière d'exhiber un contre-exemple, alors que dire OUI requière l'arbre entier.

e. Le problème CONTRADICTION qui renvoie OUI ssi la formule donnée est une contradiction est-il NP ou Co-NP ?
f. Qu'en est-il du problème TAUTOLOGIE ?

La conjecture usuellement assumée est que P≠NP, c'est-à-dire qu'il n'existe pas d'algorithme polynomial (sur architecture Von Neumann) permettant de résoudre des problèmes NP. La preuve de cette conjecture fait partie des 7 problèmes du millénaire.

3 Réduction polynomiale

Etant donné un nouveau problème, pour lequel nous décidons de proposer un algorithme optimal en temps exponentiel, ou une heuristique polynomiale, comment être sûr que l'on ne pourrait pas proposer un algorithme optimal polynomial ?

En démontrant qu'il est aussi dur qu'un problème NP-complet, nous montrons qu'il est **NP-difficile**. Si on souhaite montrer qu'il NP-complet, il faut juste montrer qu'il est NP (vérification réponse donnée en temps polynomial).

Soit le problème d'ordonnancement classique PD2|res1..|Cmax qui signifie minimiser le ***makespan*** de l'exécution de jobs sur deux processeurs dédiés, sachant que les jobs partagent une ressource. Ce problème arrive lorsqu'on rince les circuits hydrauliques d'un avion civile. Noter que ce problème n'est pas un problème de décision, sa version décisionnelle serait PD2|res1..|Cmax≤k pour un k donné.

a. Proposer une stratégie polynomiale résolvant de façon optimale PD2|res1..|Cmax

Le souci est que lorsqu'on rince des circuits hydrauliques, il y a des contraintes de précédences entre jobs de façon à ne pas re-salir un tuyau en en rinçant un autre. Le problème devient alors PD2|res1..,localprecedence|Cmax≤k qui n'avait jamais été étudié dans la littérature. Notre stratégie ne fonctionne pas, si les précédences ne sont pas consistantes avec elle… Existe-t-il une stratégie optimale polynomiale ?

Considérons le problème NP-complet 3-PARTITION : soit un ensemble de 3n entiers positifs, dont la somme totale est nB. Est-il possible de les regrouper en n triplets dont la somme individuelle est B ?

Essayons de montrer que PD2|res1..,localprecedence|Cmax≤k est aussi dur que 3PART en proposant un moyen (qui doit s'exécuter en temps polynomial) de transformer toute instance de problème 3PART en une instance de notre problème, telle que si on pouvait résoudre notre problème en temps polynomial, alors on serait capable de résoudre 3PART en temps polynomial. Cela s'appelle une **réduction polynomiale**.

Soit un problème 3SAT donné sous forme $\{i_1, i_2, etc, i_{3n}\}$ avec la somme qui vaut nB. Créons une instance de PD2|res1..|Cmax≤k de la façon suivante :

- Nous supposons deux machines, partageant une ressource r.
- Sur machine 1, nous affectons 3n jobs, de durée respective i_1, i_2, etc, i_{3n} qui utilisent tous la ressource R.
- Sur machine 2, nous affectons B jobs de durée n n'utilisant pas la ressource, et B jobs de durée 1 utilisant la ressource. On introduit des contraintes de précédence entre ces jobs pour forcer les jobs de durée 1 à s'intercaler entre les jobs de durée n.

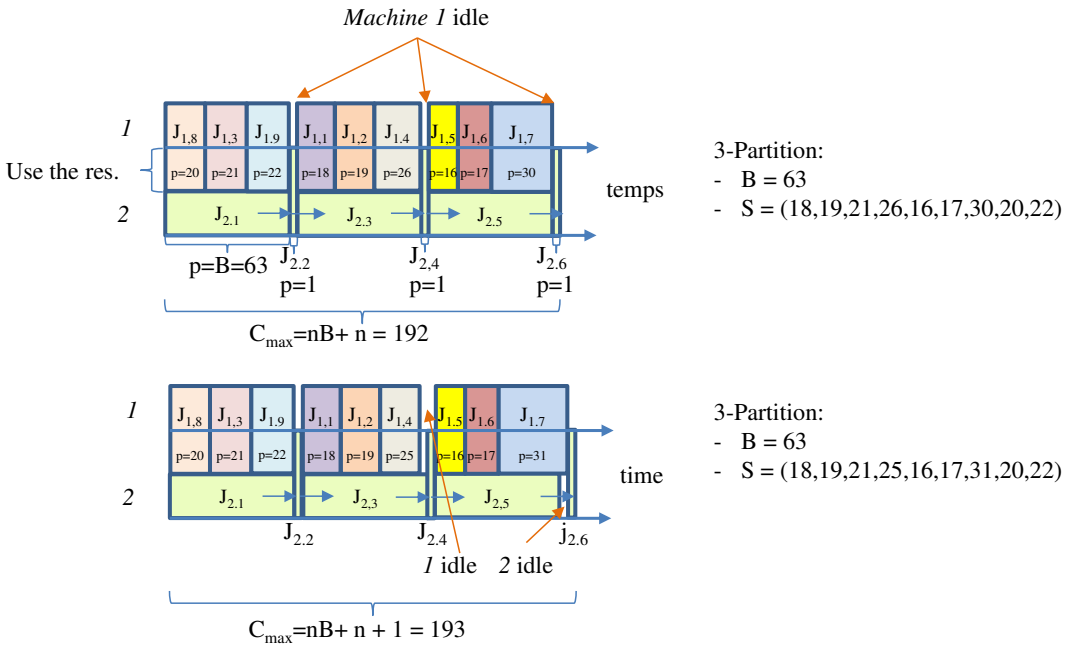


Figure 3 : Modélisation de deux problèmes 3PART sous forme de problèmes PD2|res1..|Cmax

b. Observez la Figure 3 et en déduire :

- Quel est le makespan minimal atteignable ?
- Dans quel cas peut-on l'atteindre ?
- Que peut-on en déduire sur la complexité de PD2|res1..|Cmax≤k?
- Ce problème est-il NP ?
- Que peut-on en déduire sur sa complexité ?
- Que peut-on en déduire de la complexité de PD2|res1..|Cmax ?

4 Sens fort sens faible

Dans la littérature du temps réel, on distingue deux classes de problèmes NP : sens fort et sens faible. Les problèmes NP-difficiles au sens faible sont ceux pour lesquels si l'entrée était donnée en base 1, ils seraient P. On parle de problèmes que l'on peut résoudre en temps pseudo-polynomial.

Par exemple, le crible d'Eratosthène qui permet de décomposer un nombre en facteurs premiers est pseudo-polynomial. Le calcul de temps de réponse de tâches indépendantes ordonnancées par un algorithme à priorités fixes est NP-difficile au sens faible. Vérifier le respect d'échéances d'un système de tâches sous contraintes de ressources partagées est NP-difficile au sens fort.

5 Bibliographie

[1] M. R. Garey et D. S. Johnson, *Computers and intractability: a guide to the theory of NP-completeness*. in A series of books in the mathematical sciences. New York: W. H. Freeman, 1979.

TD VAT 1 : Analyse de pire durée d'exécution

La durée d'exécution d'un programme dépend de plusieurs facteurs :

- les instructions élémentaires le composant ;
- le processeur et sa vitesse. Naïvement, on pourrait penser que chaque instruction prend 1 cycle, et donc, fréquence 2x plus élevée donne traitement 2x plus rapide. Ceci s'approche de la réalité sur des microcontrôleurs ancienne génération, mais est de plus en plus faux. C'est en tout cas totalement faux sur un microprocesseur ;
- l'état interne du processeur, qui peut par exemple contenir des pipelines ;
- l'état de la mémoire cache : l'instruction à exécuter, et les données requises, sont-elles en cache (accès 1 à 2 cycles) ou dans la RAM (accès en 3 à 10 cycles) ou stockées sur le disque dur (swap) car nous utilisons de la mémoire virtuelle (accès en plusieurs dizaines de milliers de cycles)

Il existe deux familles de méthodes pour analyser le WCET (Worst-Case Exécution Time) d'un programme sur un calculateur.

- Analyse statique de WCET, utilise un programme qui doit avoir un modèle précis du calculateur employé. Exemples : OTAWA, aiT, RapiTime, Heptane, etc. Sûre (jamais optimiste) mais nécessite un modèle précis de l'architecture d'exécution. Pessimiste lorsqu'il y a des caches et pipelines car toute inconnue sur l'état du système est modélisée de façon conservative.
- Analyse dynamique de WCET (voir Figure 1), qui consiste à instrumenter le code avec des mesures d'horloge (logicielles ou matérielles – horloges dans le CPU, analyseur logique), et mesurer de nombreuses exécutions. La difficulté est qu'il est généralement impossible de mesurer tout état interne et tout chemin (couverture du code). La solution est donc non sûre.
    - Solutions mixtes avec considération de la couverture ;
    - WCET probabiliste – distribution cumulative (cdf) de pourcentage des exécution vs. pire durée.

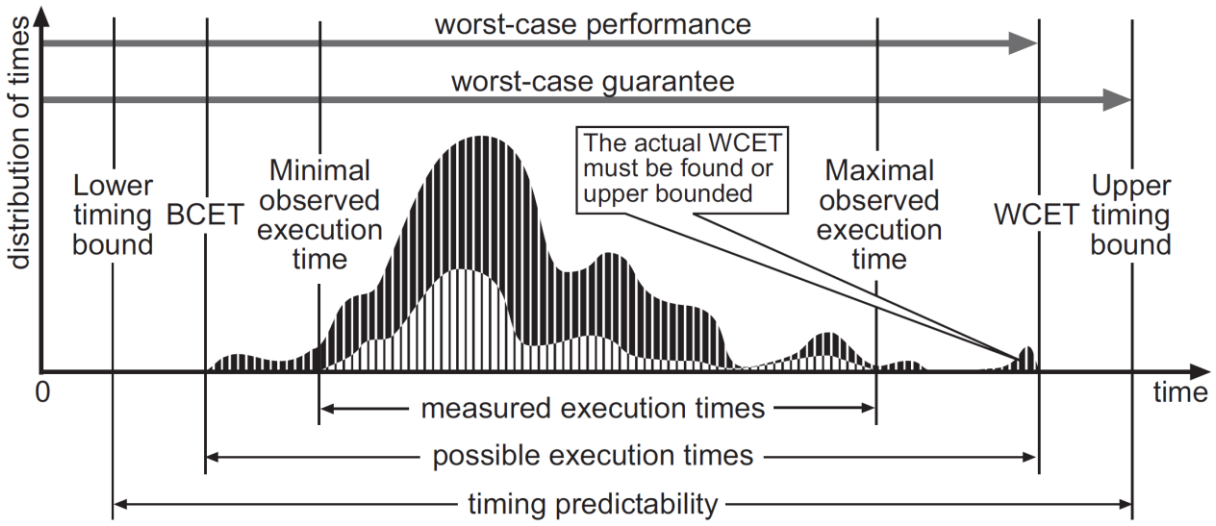


Figure 1 : Distribution typique d'une durée d'exécution, selon [1], l'un des survey les plus cités dans le domaine de la Timing Analysis (analyse de WCET)

1 Analyse statique avec OTAWA

Note : la partie du TD portant sur OTAWA a été conçue par Antoine Bertout (LIAS).

Nous allons tester OTAWA qui est un framework d'analyse statique développé à l'IRIT (Institut de Recherche en Informatique de Toulouse). Dans son principe, OTAWA décode le binaire du programme, construit son CFG (graphe de flot de contrôle) et abstrait finement le matériel. Il combine ces informations pour borner le nombre de fois que les différentes parties du programme (blocs de bases) peuvent être exécutés et le temps (nb cycles) d'exécution de chacune de ces parties. Nous ferons l'hypothèse que ce programme est destiné à être exécuté sur un microcontrôleur ARM LPC2138 de la compagnie NXP car le modèle de cette architecture est déjà disponible dans OTAWA.

1.1 Préparation

Sous Linux :

```
xhost +local:

docker run -it --rm -e DISPLAY=:0.0 --net=host registry-
ia.ensma.fr/svtr /bin/bash
```

Sous Windows avec WSL2 :

```
docker run -it --rm -v /tmp/.X11-unix:/tmp/.X11-unix -e DISPLAY -e
WAYLAND_DISPLAY -e XDG_RUNTIME_DIR -e PULSE_SERVER --net=host
registry-ia.ensma.fr/svtr /bin/bash
```

1.2 Utilisation d'OTAWA

Plusieurs programmes C sont disponibles dans le répertoire wcet, comme bs.c avec une simple fonction main qui appelle la fonction de recherche dichotomique binary_search dans un tableau.

1. Commencer par visualiser le programme.

```
cd wcet ; cat bs.c
```

2. Compiler le programme avec gcc pour architecture ARM (on parle de compilation croisée)

```
arm-linux-gnueabi-gcc -g3 -fno-stack-protector -marm -march=armv4 -o
bs.elf bs.c -nostartfiles -static -T link-flash.ld;
```

3. Lancer l'estimation de WCET de la fonction main avec OTAWA.

```
owcet -s lpc2138 bs.elf
```

En essayant d'interpréter la sortie de owcet, on peut comprendre qu' OTAWA a besoin de connaître le nombre maximal de fois que la boucle de la fonction binary_search peut être appelée (borne de boucle). Quelle est cette borne N (pour rappel, le tableau possède 15 éléments) ?

4. Nous allons préciser cette borne manuellement en éditant (avec vim) un fichier ff (flowfact) produit par la commande suivante.

```
mkff bs.elf > bs.ff
```

Dans bs.ff remplacer le symbole ? par la valeur de N trouvée précedemment. Dans vim, taper i pour passer en mode édition, échap pour sortir du mode édition, et :wq pour sauvegarder et quitter vim.

5. Relancer owcet en liant le fichier de flow fact et obtenir le WCET du programme sur le microcontrôleur.

```
owcet -s lpc2138 bs.elf -f bs.ff
```

6. Fort heureusement, l'outil orange d'OTAWA permet de déterminer automatiquement cette borne de boucle, vérifiez la valeur trouvée dans le fichier ffx en comparant avec la vôtre.

```
orange bs.c main -o bs.ffx
```

7. Nous allons générer un ensemble de statistiques qui vont nous permettre de visualiser le CFG du programme annoté.

```
owcet -s lpc2138 bs.elf --stats -S
```

8. Lancer l'interface graphique de visualisation du CFG.

```
obviews.py bs main
```

9. Recompilez le programme avec l'option -O1, relancez la commande owcet du 7. Et observez les différences de WCET.

2 Analyse dynamique

Télécharger bs.c sur votre machine, à partir du dossier Resources du TD sur Moodle. La fonction de recherche dichotomique est la même que celle que nous avons mesurée statiquement pour la cible LPC2138. Supposons que le programme doive s'exécuter sur le CPU de votre machine. Il est probable que le modèle de votre CPU ne soit pas disponible pour un analyseur de WCET statique. Nous allons donc effectuer des mesures dynamiques.

1. Compiler et lancer plusieurs fois le programme. Comment peut-on expliquer les variations observées ?

2. Modifier la valeur d'indice de fin de boucle for, en la passant à 10, 100, 1000. Comment expliquez-vous la variation décroissante de la moyenne observée de chaque durée d'exécution ?

3 Bibliographie

[1] R. Wilhelm *et al.*, « The worst-case execution-time problem—overview of methods and survey of tools », *ACM Trans. Embed. Comput. Syst.*, vol. 7, n° 3, p. 36:1-36:53, mai 2008, doi: 10.1145/1347375.1347389.

TD VAT 2
Validation par simulation et viabilité

La simulation consiste à tracer le fonctionnement attendu de l'ordonnancement d'un système. Elle ne permet de représenter qu'un comportement unique : pour une période donnée pour chaque tâche, une durée donnée, une date de réveil donnée. Cependant, nous avons vu que la durée d'exécution en-ligne était variable, qu'il existait des taches sporadiques dont la période représente en fait le délai minimal entre deux activations successives, et que certains systèmes sont concrets (dates de réveil connues) alors que d'autres sont non-concrets (dates de réveil inconnues).

La simulation permet de valider un système si et seulement si elle représente le comportement pire cas du système. Cela n'est vrai que si la simulation est **viable** dans le **contexte** pour les paramètres qui ne sont pas fixés.

Contexte : hypothèses s'appliquant au système étudié. Par exemple : « ordonnanceur en-ligne sur un système monoprocesseur, tâches périodiques simultanées indépendantes, et à échéances sur requête. »
Quelques éléments liés au contexte :
Processeur : monoprocesseur, multiprocesseur global, multiprocesseur partitionné, réparti
Ordonnanceur : en-ligne ou hors-ligne
Taches : concrètes ou non
Tâches concrètes : simultanées ou non
Activations des tâches : périodiques, sporadiques, apériodiques, reliées
Interdépendances des tâches : indépendantes, avec sections critiques (ou portions non préemptibles), contraintes de précédence
Structure interne des tâches : avec ou sans suspension

Viabilité :
Un test d'ordonnançabilité est **C-viable** dans un contexte si réduire la durée d'exécution ne peut augmenter de temps de réponse.
Il est **T-viable** dans un contexte si augmenter une période ne peut augmenter un temps de réponse.
Il est **r-viable** si augmenter une date de réveil ne peut augmenter un temps de réponse.

Exercice 1 :

Soit un système de tâches indépendantes, périodiques, ordonnancé en-ligne par un ordonnanceur à priorités fixes aux tâches sur un processeur, caractérisé par :

	Date de réveil r_i	**WCET C_i**	**Délai critique D_i**	**Période T_i**	**Priorité Π_i**
τ_1	0	8	30	30	1
τ_2	0	6	20	20	2
τ_3	0	4	15	15	3

L'**utilisation processeur** $U = \sum_{i=1}^{n} \frac{C_i}{T_i}$ (aussi appelée charge processeur) d'un système fournit une condition nécessaire triviale d'ordonnançabilité. Il faut en effet que U≤k pour qu'un système ait une

chance d'être ordonnançable sur un système de k processeurs. Si U>1, le système est trivialement non-ordonnançable sur 1 processeur.

a) Calculer l'utilisation processeur (aussi appelée charge processeur) U du système, et conclure.
b) Tracer la simulation de l'ordonnancement du système, puis vérifier que la séquence produite est la même que celle produite par SimSo Web (http://projects.laas.fr/simso/simso-web/#/configuration).
c) Quel est le pire temps de réponse observé de chaque tâche ? Chaque tâche respecte-t-elle toutes ses échéances ?

Lorsque les tâches sont indépendantes, préemptibes, et ordonnancées sur un processeur, la **simulation est C-viable et T-viable**.
De plus, si le système est non concret, la simulation du système de tâches en considérant des réveils simultanés est **r-viable**.

d) Le système est-il ordonnançable par Rate Monotonic ?
e) En supposant que les dates de réveil soient inconnues, le système est-il ordonnançable ?
f) Tracer la simulation de l'ordonnancement du système en supposant cette fois un ordonnanceur Earliest Deadline First, puis vérifier que la séquence produite est la même que celle produite par SimSo Web.
g) Quel est le pire temps de réponse de chaque tâche ?
h) Le système est-il ordonnançable par EDF ?

Exercice 2 :

Considérons un système de tâches différées, dont les paramètres sont donnés ci-après :

	Date de réveil r_i	**WCET C_i**	**Délai critique D_i**	**Période T_i**	**Priorité Π_i**
τ_1	8	8	30	30	1
τ_2	4	6	20	20	2
τ_3	0	6	15	15	3

a) Calculer l'utilisation processeur (aussi appelée charge processeur) U du système, et conclure.

Si le système est ordonnançable, en monoprocesseur, avec $D_i \leq T_i$, la première fenêtre de taille H=PPCM(T_i) contenant exactement H(1-U) temps creux donne le cycle de l'ordonnancement. Ceci arrive au plus tard à la date max(r_i)+2H.

b) Tracer la simulation de l'ordonnancement du système, puis vérifier que la séquence produite est la même que celle produite par SimSo Web.
c) Tracer la simulation en supposant que les tâches sont simultanées, et conclure.

Pour une tâche de priorité Π_i, le cas le plus défavorable survient lorsqu'elle subit la pire interférence de toutes les tâches de priorité plus élevée. L'**instant critique** d'une tâche survient lorsqu'elle est activée simultanément avec toutes les tâches plus prioritaires, car cela maximise l'interférence de celles-ci.

d) En supposant les dates de réveil données dans la table, mais à supposer cette fois que les tâches sont sporadiques, que conclure ?

Exercice 3 :

Nous considérons un système de tâches pour lesquelles τ_3 précède τ_1.

	Date de réveil r_i	WCET C_i	Délai critique D_i	Période T_i	Priorité Π_i
τ_1	0	8	12	12	3
τ_2	1	2	3	12	2
τ_3	0	2	12	12	1

a) Tracer la simulation de l'ordonnancement produit.
b) Supposer maintenant que C_3=1, tracer la simulation.
c) Que conclure sur la viabilité de la simulation lorsqu'il existe des contraintes de précédence ?

Exercice 4 :

Nous considérons un système de tâches pour lesquelles τ_1 et τ_2 sont en exclusion mutuelle pendant toute leur durée. Nous supposons une affectation de priorités Deadline Monotonic (priorité inversement proportionnelle au délai critique).

	Date de réveil r_i	WCET C_i	Délai critique D_i	Période T_i
τ_1	0	6	16	16
τ_2	0	2	6	8
τ_3	0	6	15	16

a) Tracer la simulation de ce système.
b) Tracer une simulation lorsque C_3=5
c) Que conclure sur la viabilité de la simulation lorsqu'il existe des portions de tâches non préemptibles ?

TD VAT 3
Optimalité en FTP

La simulation, pour des systèmes de tâches indépendantes :

- Est une CNS (Condition Nécessaire et Suffisante) d'ordonnançabilité pour les systèmes de tâches périodiques ou sporadiques en monoprocesseur ;
- Est une CNS (Condition Nécessaire et Suffisante) d'ordonnançabilité pour les systèmes de tâches périodiques en multiprocesseur ;

Cependant, la simulation possède deux inconvénients majeurs :

- Dès lors qu'il existe des facteurs pratiques (exclusions mutuelles, parties non préemptibles, suspension, contraintes de précédence non consistantes avec les priorités), la simulation perd toute propriété de viabilité. Au mieux, elle peut être utilisée pour trouver un contre-exemple, ou bien une séquence hors-ligne à exécuter en-ligne, mais en aucun cas elle ne peut être utilisée pour valider un système. Elle devient dans ce cas une condition suffisante de non-ordonnançabilité (i.e. elle peut exposer un contre-exemple, mais ne l'expose pas forcément s'il existe).
- La simulation possède une complexité exponentielle, d'après (Chan & Norrish, 2016), PPCM(1, 2,…n+1) ≥ 2^n.

Plusieurs auteurs ont proposé des tests d'ordonnançabilité de complexité polynomiale. Cependant, on sait que la complexité du problème d'ordonnançabilité est Co-NP-difficile au sens fort s'il existe des dates de réveil d'une part (Leung & Whitehead, 1982), et que la complexité du problème de calcul de temps de réponse est NP-difficile au sens faible (pseudo-polynomial) même lorsque les tâches sont réveillées simultanément (Eisenbrand & Rothvos, 2008). Par conséquent, un algorithme de complexité polynomiale ne peut pas être exact. Les tests proposés sont donc généralement des heuristiques conservatives, en d'autres termes des conditions suffisantes (mais non nécessaires) d'ordonnançabilité. De plus, elles adressent une affectation de priorités particulière, en l'occurrence elles adressent les affectations de priorité optimales.

Rate Monotonic (RM), qui affecte les priorités de façon inversement proportionnelle à la période des tâches est optimale dans la classe des algorithmes à priorités fixes aux tâches lorsque (Liu & Layland, 1973) :

1. Toutes les tâches sont indépendantes préemptibles ;
2. Et le système est monoprocesseur ;
3. Et il existe un instant critique (i.e. les tâches peuvent être activées simultanément) ;
4. Et les tâches sont périodiques ou sporadiques (dans le cas sporadique, la condition 3 est forcément satisfaite) ;
5. Et Les tâches sont à échéance sur requête (i.e. $D_i = T_i$).

Un algorithme d'ordonnancement A est **optimal** dans une classe d'algorithmes d'ordonnancement Ψ, si $A \subset \Psi$ et si $\exists B \subset \Psi$ tel que B ordonnance fiablement un système S, alors A ordonnance fiablement S.

Les deux tests polynomiaux les plus connus pour RM sont :

La **borne de Liu & Layland** (Liu & Layland, 1973) : un système de n tâches indépendantes à échéances sur requête est fiablement ordonnancé par RM sur un processeur si $U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq n\left(\sqrt[n]{2} - 1\right)$. De

plus, $\lim_{n\to\infty}\left(n\left(\sqrt[n]{2}-1\right)\right) = \ln 2 \approx 0{,}693147$. Cela implique notamment qu'un système de tâches est ordonnançable (pour sûr) par RM si sa charge est inférieure ou égale à 69%.

La **borne hyperbolique** (Bini, Buttazzo, & Buttazzo, 2003) : soit la charge unitaire d'une tâche définie par $U_i = \frac{C_i}{T_i}$. Un système de n tâches indépendantes à échéances sur requête est fiablement ordonnancé par RM sur un processeur si $\prod_{i=1}^{n}(U_i + 1) \leq 2$. La borne hyperbolique est strictement plus puissante que le test de L&L, cela signifie que tout système validable par L&L est aussi validable par la borne hyperbolique, mais qu'il existe des systèmes validables par la borne hyperbolique qui ne passent pas le test de L&L. Cela rend théoriquement le test de L&L obsolète, cependant, comme on calcule toujours la charge d'un système pour vérifier qu'il n'est pas trivialement non ordonnançable, le test de L&L est immédiat, ce qui le rend tout de même intéressant à connaître.

Dans le cas où $D_i \leq T_i$, ces tests s'appliquent en remplaçant la charge U_i par la densité C_i/D_i. Ils deviennent cependant dans ce cas encore plus pessimistes que les tests à échéance sur requête.

Exercice 1 :

Soit un système de tâches indépendantes, périodiques, ordonnancé en-ligne par un ordonnanceur à priorités fixes aux tâches sur un processeur, caractérisé par :

	Date de réveil r_i	WCET C_i	Délai critique D_i	Période T_i
τ_1	0	8	30	30
τ_2	0	6	20	20
τ_3	0	4	15	15

a) Calculer l'utilisation processeur (aussi appelée charge processeur) U du système, et conclure.
b) RM donne-t-il une affectation de priorités fixes aux tâches optimale ?
c) Le système est-il démontré ordonnançable par un test de complexité polynomiale que vous connaissez ?
d) Regarder l'exercice 1 dans le TD de simulation et conclure.

Exercice 2 :

Considérons un système de tâches différées, indépendantes et périodiques, ordonnancé en-ligne par un ordonnanceur à priorités fixes aux tâches sur un processeur, caractérisé par :

	Date de réveil r_i	WCET C_i	Délai critique D_i	Période T_i
τ_1	8	8,05	30	30
τ_2	4	4,5	20	20
τ_3	0	4,3	15	15

a) Calculer l'utilisation processeur (aussi appelée charge processeur) U du système, et conclure.
b) RM donne-t-il une affectation de priorités fixes aux tâches optimale ?
c) Peut-on tout de même appliquer le test de L&L et le test hyperbolique ? Si oui, les appliquer et conclure.

Deadline Monotonic (DM), qui affecte les priorités de façon inversement proportionnelle au délai critique est optimale dans la classe des algorithmes à priorités fixes aux tâches lorsque (Leung & Whitehead, 1982) :

1. Toutes les tâches sont indépendantes préemptibles ;

2. Et le système est monoprocesseur ;
3. Et il existe un instant critique (i.e. les tâches peuvent être activées simultanément) ;
4. Et les tâches sont périodiques ou sporadiques (dans le cas sporadique, la condition 3 est forcément satisfaite) ;
5. Et Les tâches sont à **échéance contrainte** (i.e. $D_i \leq T_i$).

En fait, on peut voir RM comme le cas particulier de DM pour les tâches à échéances sur requêtes.

Exercice 3 :

Soit un système de tâches indépendantes, **sporadiques**, ordonnancé en-ligne par un ordonnanceur à priorités fixes aux tâches sur un processeur, caractérisé par :

	WCET C_i	**Délai critique D_i**	**Période T_i**
τ_1	4	16	16
τ_2	3	7	8
τ_3	4	6	12

a) Connaissez-vous une affectation de priorités fixes aux tâches optimale dans ce contexte ?
b) Appliquer les tests de L&L et hyperbolique en considérant la densité, et conclure.
c) Tracer une simulation du système suivant cette affectation de priorités.
d) Existe-t-il une affectation de priorités fixes aux tâches ordonnançant fiablement ce système ?

Exercice 4 :

Lorsqu'il existe des délais critiques plus grand que la période, ou bien lorsque certaines tâches sont différées et qu'il n'y a pas d'instant critique, RM et DM perdent leur optimalité. Il existe un moyen permettant de tester efficacement toute affectation de priorités possible, si le contexte satisfait 3 conditions :

- Condition 1 : l'ordonnançabilité d'une tâche dépend des tâches plus prioritaires, individuellement, mais pas de leur ordre de priorités entre elles.
- Condition 2 : l'ordonnançabilité d'une tâche dépend des tâches moins prioritaires, individuellement, mais pas de leur ordre de priorités entre elles.
- Condition 3 : si une tâche respecte ses échéances avec un niveau de priorité *i*, elle ne peut pas ne pas les respecter avec un niveau de priorité *i+1*.

Si ces 3 conditions sont satisfaites par le test d'ordonnançabilité employé, alors on peut utiliser l'algorithme d'Audsley (Audsley, 1991), dit « Optimal Priority Assignment » ou OPA.

```
T=liste des tâches (triée par exemple par ordre croissant DM)
Pour priorité Π=1 à n
      τ=tete(T)
      Prio_affectée=faux
      Répéter
            Si τ ne respecte pas ses échéances avec priorité Π
                  Si τ=last(T) -- on a épuisé toute la liste
                        Renvoyer non ordonnançable
                  Sinon τ=next(T)
            Sinon
                  Prio_affectée=vrai
                  Assigner priorité Π à τ et remove(T,τ)
      Jusqu'à Prio_affectée
Fait
```

a) Quelle est la complexité de cet algorithme en supposant que vérifier qu'une tâche respecte ses échéances ait une complexité O(1) ?
b) Supposons que nous ayons 25 tâches, combien au maximum de test d'ordonnançabilité de tâche devrons-nous mener ?

Une façon de tester si une tâche respecte ses échéances, indépendamment des autres priorités, consiste à simuler (si on se trouve dans un contexte ou la simulation est viable) le système en affectant :

- La priorité Π à τ
- La priorité $\Pi+1$ à toutes les autres tâches encore présentes dans T
- La priorité déjà choisie pour les tâches affectées précédemment

On ne regarde alors que pour la tâche τ considérée si les échéances sont respectées (on ne regarde pas si les échéances des tâche plus prioritaires sont respectées ou non).

c) En supposant que le test d'ordonnançabilité de tâche retenu soit la simulation, est-ce que, si les tâches sont non préemptibles, on peut utiliser OPA ?
d) Observer le système de tâches sporadiques indépendantes et préemptibles suivant. DM est-il optimal dans ce cadre ? OPA s'applique-t-il en considérant la simulation pour valider l'ordonnançabilité des tâches ? Le système est-il ordonnançable en priorités fixes aux tâches ?

	WCET C_i	**Délai critique D_i**	**Période T_i**
τ_1	4	16	16
τ_2	2	9	8
τ_3	5	13	12

Bibliographie

Audsley, N. C. (1991). *Optimal priority assignment and feasibility of static priority tasks with arbitrary start times.* Computer Science Report YCS 164, York.

Bini, E., Buttazzo, G. C., & Buttazzo, G. M. (2003). Rate monotonic analysis: the hyperbolic bound. *IEEE Transactions on Computers 52(7)*, 933-942.

Chan, H.-L., & Norrish, M. (2016). Proof pearl: bounding least common multiples with triangles. *International Conference on Interactive Theorem Proving*, (pp. 140-150).

Eisenbrand, F., & Rothvos, T. (2008). Static-priority real-time scheduling: Response time computation is NP-hard. *Real-Time Systems Symposium*, (pp. 397-406).

Leung, J. Y.-T., & Whitehead, J. (1982). On the complexity of fixed-priority scheduling of periodic, real-time tasks. *Performance evaluation*, 237-250.

Liu, C. L., & Layland, J. W. (1973). Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM 20 (1)*, 46-61.

TD VAT 4
Analyse de temps de réponse- RTA

Jusqu'à présent nous avons utilisé la simulation, de complexité exponentielle pour valider des systèmes de tâches. Nous avons vu que dans certains cas très restreints, il existait des conditions suffisantes de complexité polynomiale applicables.

Cependant, nous avons vu lors du dernier TD que la complexité de calcul de pire temps de réponse est NP-difficile au sens faible, par conséquent, il devrait exister un algorithme de calcul de temps de réponse de complexité pseudo-polynomiale.

Période d'activité de niveau i : dans un ordonnancement à priorités fixes aux tâches, une période d'activité de niveau i représente une occupation continue du processeur en ne considérant que les tâches de priorité ≥ i. Une période d'activité commence et se termine sur un point creux.

Point creux : instant pendant lequel le processeur n'a aucune charge à traiter.

La Figure 1 représente des périodes d'activité de niveau 1 (plus basse priorité) de l'ordonnancement du système de deux tâches périodiques simultanées $\{\tau_1 <C_1=1, T_1=4>, \tau_2 <C_2=10, T_2=14>\}$. Noter que les périodes d'activité sont indépendantes des échéances.

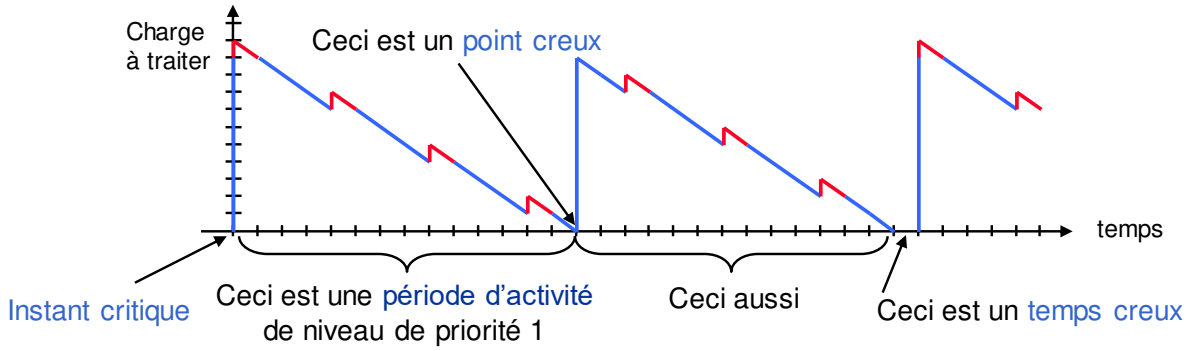


Figure 1 : Périodes d'activité, points creux et temps creux

A chaque réveil, on observe une montée de la charge à traiter (à chaque réveil de τ_i, la charge à traiter augmente de C_i). A chaque unité de temps, le processeur traite une unité de charge, si celle-ci est non nulle.

Théorème de l'instant critique : étant donné un ensemble de tâches ordonnancées en priorités fixes aux tâches, la plus longue période d'activité s'observe à l'instant critique.

Théorème (Audsley, 1991) : le plus long temps de réponse d'une tâche de priorité i se trouve dans la plus longue période d'activité de niveau i.

Sur la Figure 1, la première période d'activité dure 14 unités de temps, la seconde en dure 13. Supposons que la priorité de τ_2 soit égale à 1, cela signifie qu'elle se termine « en dernier », c'est-à-dire au point creux marquant la fin de chaque période d'activité. Elle se termine donc à l'instant 14 lors de son instance démarrée à l'instant critique, sa première instance possède donc un temps de réponse de 14 unités de temps. Sa seconde instance est activée à l'instant 14 et termine à l'instant 27,

ce qui lui donne un temps de réponse de 13 unités de temps. Ensuite, le système répète le cycle [0..28[à l'infini. Nous pouvons observer cela sur une simulation donnée sur la Figure 2.

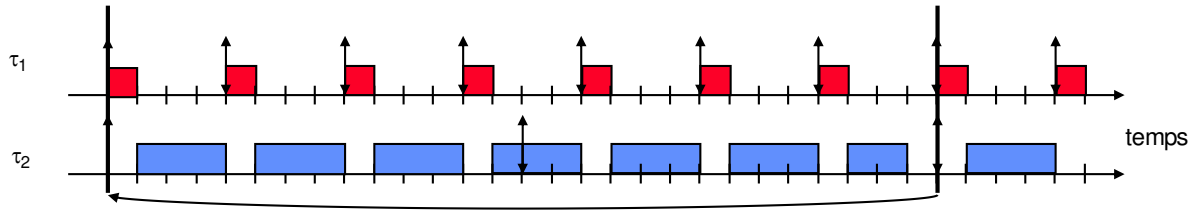


Figure 2 : Simulation de l'ordonnancement RM du même système

Pour le moment, tracer les périodes d'activité et simuler revient au même, cependant, il y a ici un premier gain : d'après le théorème d'Audsley, s'il existe un instant critique, c'est forcément dans la période d'activité initiée par l'instant critique que l'on trouve le plus long temps de réponse. On pourrait s'arrêter ici de construire la période d'activité lorsque l'on trouve le point creux à l'instant 14. En effet, dans la seconde période d'activité, le temps de réponse trouvé ne peut pas être plus long.

Cela reste cependant de complexité exponentielle, la période d'activité pouvant au pire durer le PPCM des périodes si U=1. D'après (Eisenbrand & Rothvos, 2008), on doit pouvoir trouver un algorithme plus efficace que la construction systématique de toute la période d'activité.

L'idée proposée par (Joseph & Pandya, 1986) est la suivante : soit une tâche τ_i dont on veut calculer le pire temps de réponse, et soit hp(i) l'ensemble des indices de tâches plus prioritaires que (ou ex-aequo avec) la tâche étudiée τ_i. Nous allons tenter d'établir de façon efficace la date de fin de période d'activité initiée par un instant critique ayant eu lieu à la date 0.

La période d'activité de niveau de priorité de τ_i dure au moins C_i, la pire durée de τ_i. Nous notons cette durée $w_i^0 = C_i$.

Puisque la période d'activité initiée par l'instant critique dure au moins w_i^0, toute tâche plus prioritaire activée dans l'intervalle [0.. w_i^0[va venir préempter la tâche τ_i, rallongeant ainsi la période d'activité (comme les « pointes » observées à chaque réveil de τ_i sur la Figure 1). Su l'intervalle [0..t[, le nombre de réveils d'une tâche j est donné par $\left\lceil \frac{t}{T_j} \right\rceil$, à chacun de ces réveils, la période d'activité est prolongée de C_j. Par conséquent, si la période d'activité dure au moins w_i^0, alors elle dure au moins :

$$w_i^1 = C_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^0}{T_j} \right\rceil C_j$$

Mais si elle dure au moins w_i^1, alors tout réveil d'une tâche plus prioritaire sur [0.. w_i^1[vient allonger cette période d'activité. On voit que l'on recherche en fait le plus point fixe non nul w_i^* tel que :

Response-Time Analysis (RTA): $w_i^* = C_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^*}{T_j} \right\rceil C_j$

w_i^* donne la date de fin de la première instance de la tâche τ_i initiée par l'instant critique. Le pire temps de réponse de la première instance de cette tâche, activée à l'instant 0, est donc $TR_i(1) = w_i^*$.

- Si ce temps de réponse n'est pas supérieur à la période de τ_i, alors il correspond à la date de fin de la période d'activité, qui ne contient par conséquent qu'une instance de τ_i. D'après le théorème d'Audsley, c'est donc le pire temps de réponse de la tâche et $TR_i = TR_i(1)$. La complexité de ce calcul de point fixe est pseudo-polynomiale.

- Si $w_i^* > T_i$, nous n'avons pas fini. Cela signifie que la seconde instance de τ_i appartient aussi à la période d'activité la plus longue. Par conséquent, cette instance peut aussi être celle qui possède le plus long temps de réponse, et il faut donc l'étudier.

C'est ce que propose (Lehoczky, 1990) en proposant une formule généralisée du calcul de RTA :

$k = 0$
Répéter
    $k = k + 1$
    $w_i^0(k) = kC_i$
    Trouver le plus petit point fixe non nul $w_i^*(k) = kC_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^*(k)}{T_j} \right\rceil C_j$
    $TR_i(k) = w_i^*(k) - (k-1)T_i$
Jusqu'à $TR_i(k) \leq T_i$
$TR_i = \max_{j=1..k}\left(TR_i(j)\right)$

Cette formule est **à connaître sur le bout des doigts**, puisqu'elle est à la base de l'analyse de temps de réponse en priorités fixes aux tâches. C'est une formule de calcul exacte de temps de réponse, de complexité pseudo-polynomiale. Elle donne une **condition nécessaire et suffisante** d'ordonnançabilité lorsqu'il existe (ou peut exister) un instant critique, puisqu'il suffit de vérifier que pour toute tâche, $TR_i \leq D_i$.

Remarquer que pour k=1, cette formule est celle de (Joseph & Pandya, 1986).

Nous verrons par la suite des extensions de cette formule pour prendre en compte les exclusions mutuelles et les contraintes de précédence.

Exercice 1

Lien entre formule RTA et représentation de la période d'activité.

Observer la période d'activité du système de deux tâches {τ_1 <C_1=1, T_1=4>, τ_2 <C_2=10, T_2=14>} représentée sous forme de fonction escalier, appelée RBF (pour Request Bound Function) sur la Figure 3.

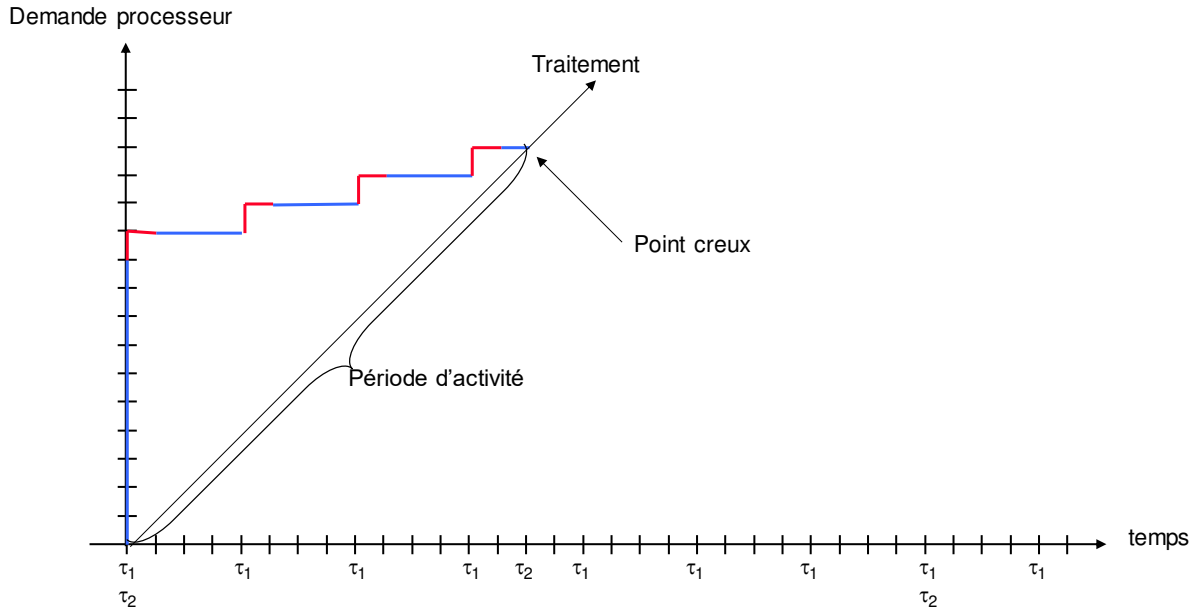


Figure 3 : Période d'activité représentée sous forme de RBF

Calculer avec la RTA le pire temps de réponse de la tâche τ_2, et observer la représentation graphique des pas de calcul la Figure 4.

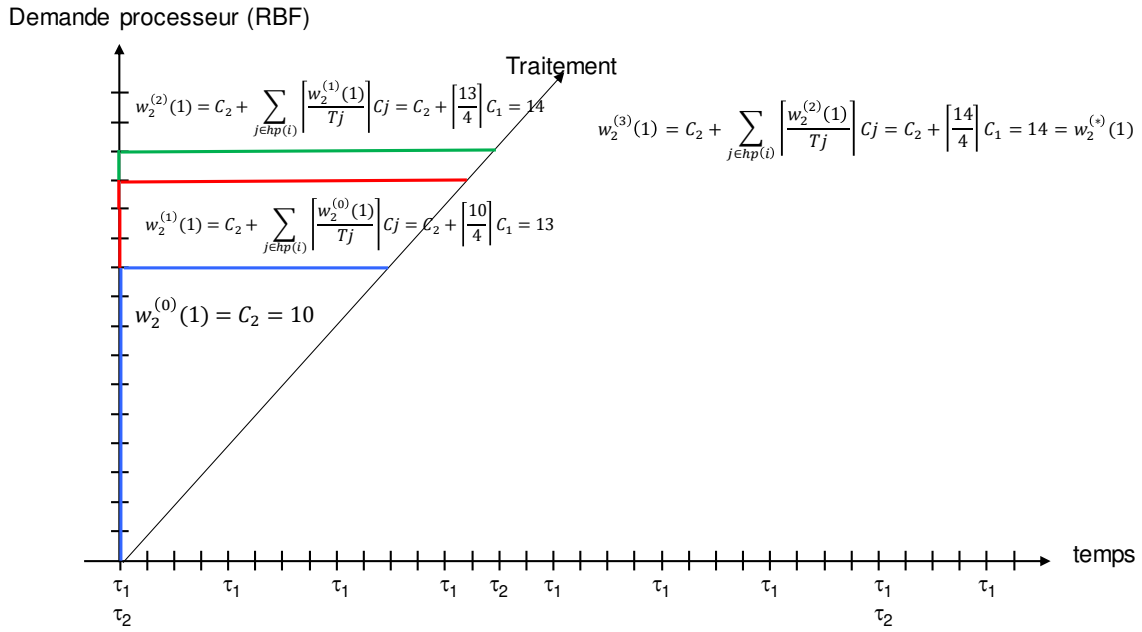


Figure 4 : Valeurs obtenues par le calcul RTA représentées sur la RBF

Le calcul RTA permet donc de calculer de façon efficace quelle est l'intersection entre la RBF et la puissance du processeur, représentée par la droite y=x.

Exercice 2

Soit le système S1 de trois tâches indépendantes, périodiques, simultanées, à échéances sur requête, dont les paramètres <C_i, T_i> sont donnés par {<2,4>, <1,4>, <1,8>}, ordonnancé suivant RM.

a) Calculer la charge du système S1, et conclure.
b) RM est-il optimal pour S1 ?
c) La RTA est-elle exacte, pessimiste ou optimiste ?
d) Calculer avec la RTA le temps de réponse de chaque tâche, et conclure.

Exercice 3

Soit le système S2 de trois tâches indépendantes, sporadiques, à échéances sur requête, dont les paramètres <C_i, T_i> sont donnés par {<2,7>, <2,4>, <3,14>}, ordonnancé suivant RM.

a) Calculer la charge du système S2, et conclure.
b) RM est-il optimal pour S2 ?
c) La RTA est-elle exacte, pessimiste ou optimiste ?
d) Calculer avec la RTA le temps de réponse de chaque tâche, et conclure.

Exercice 4

Soit le système S3 de trois tâches indépendantes, périodiques, à échéances sur requête, à départs différés, dont les paramètres <O_i, C_i, T_i> sont donnés par {<0,1,4>, <1,3,6>, <2,1,4>}, ordonnancé suivant RM.

a) Calculer la charge du système S3, et conclure.